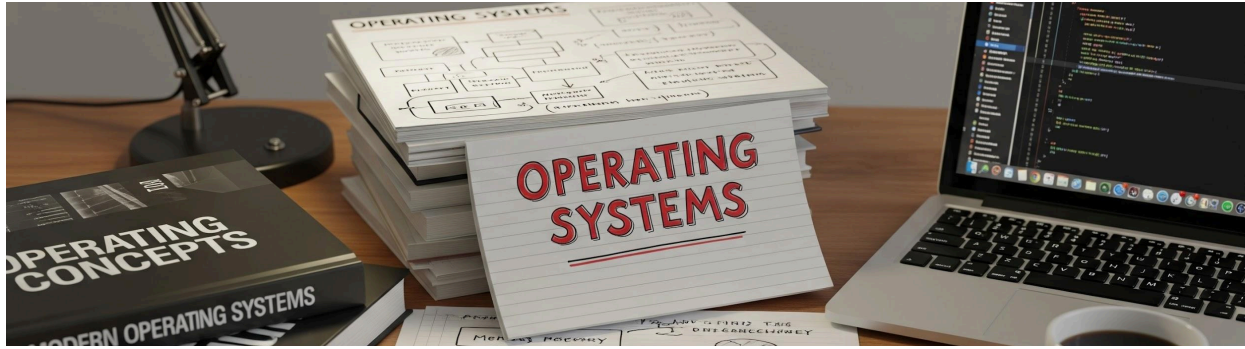


Operating systems are the best example of an abstract machine.

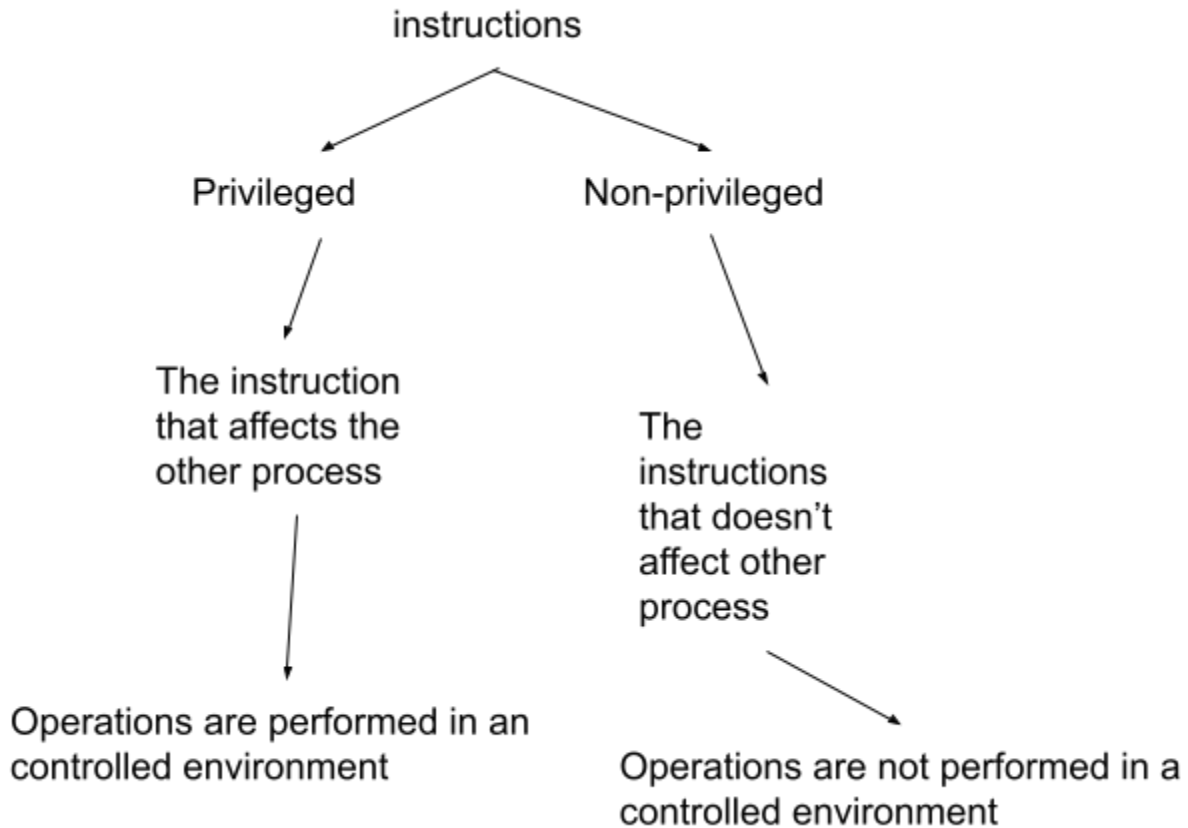
Operating system is an:

- *Resource manager*
- *Control program*
- *Service provider*

There is no proper definition for operating systems as we can define it by how we look at it (or) the perspective of the operating system by the users.

The types of users are:

- *Developers*
- *End-users*
- *Administrators (OS as resource manager)*



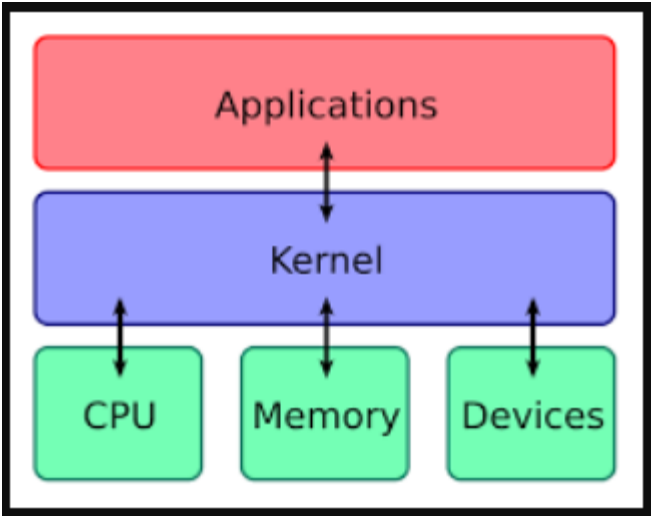
The operating system has two modes (dual mode) namely:

- Kernel mode
- User mode

User space: range of address allocated to the user

Kernel space: range of address allocated to the kernel

The kernel is the heart of the system and in all OS, the kernel is always the Linux kernel eg: windows and macOS.



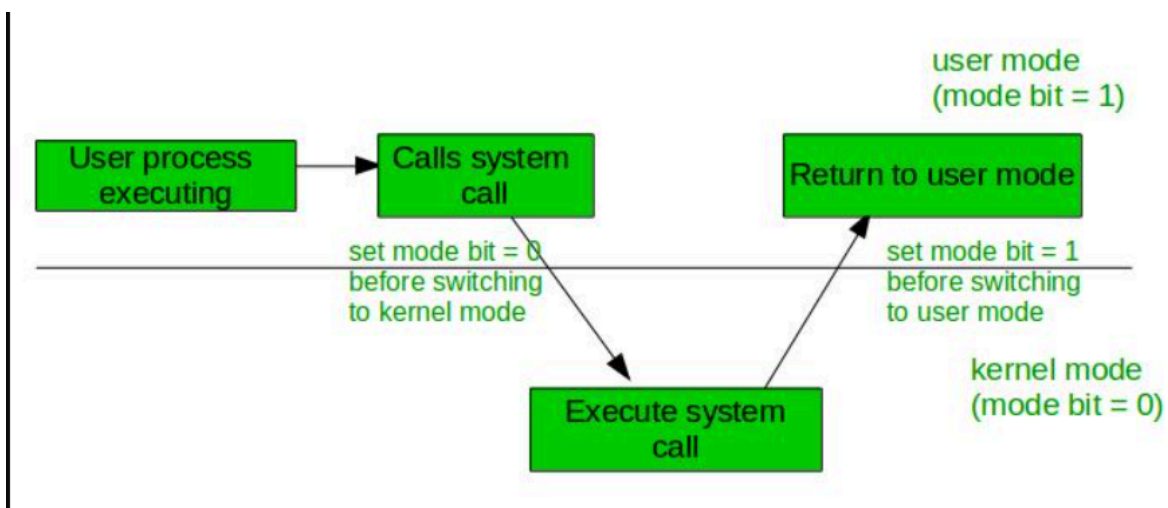
From the above diagram it is clear that the kernel acts as the gateway to communicate with the application including the OS.

In the PSW register there is a bit used as mode bit.

If mode bit == 1 $\Rightarrow$ user mode

If mode bit == 0 $\Rightarrow$ kernel mode

- In kernel mode the program has full power as it can access any resource.



System calls are like the API calls provided by the operating system.

Functionalities of OS:

- *Resource management*
- *Security/ error handling /recovery /protection*
- *Program execution*
- *IO management*
- *File system*
- *User management*
- *Networking & communication*
- *Accounting*

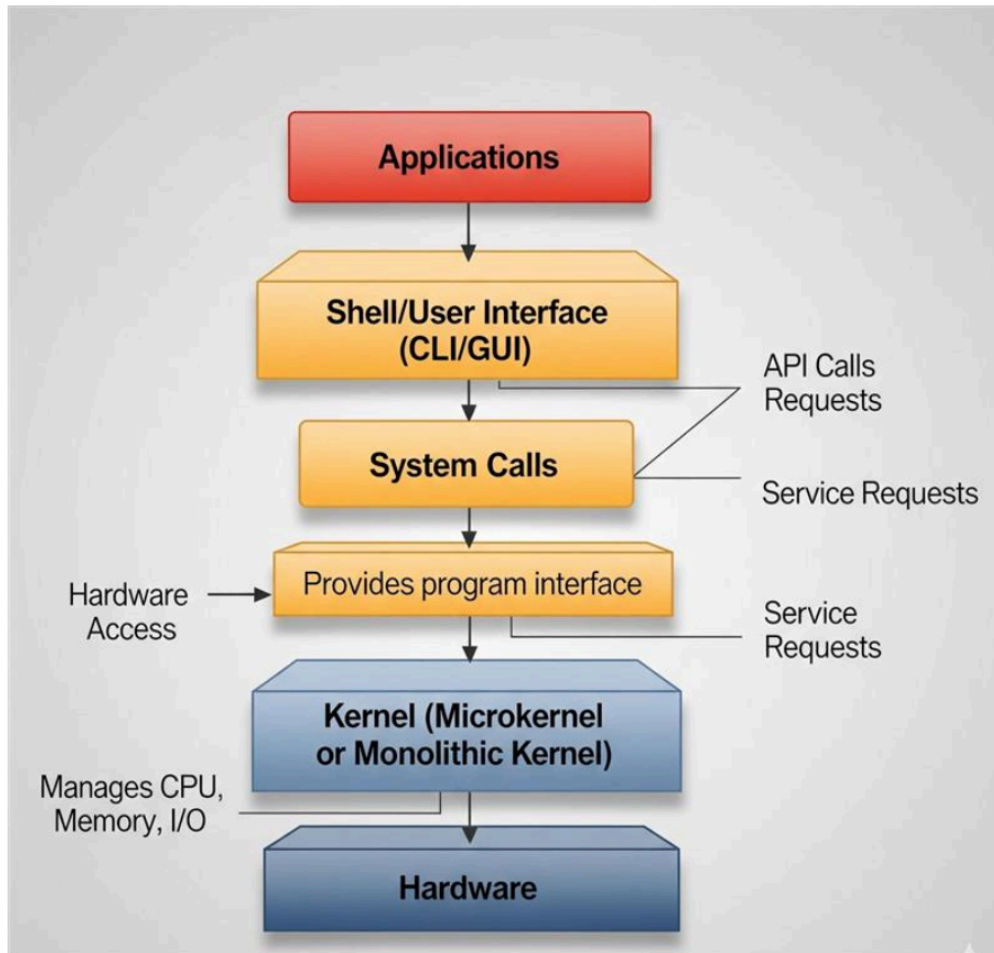
The client server architecture that client and server both will have different programs running in which the client will be the one to communicate with the service.

- *The browser is a client program. The program that runs in the machine is called the client machine.*
- *The machine that runs the server program is known as the server .*

types of interface :

- *GUI - graphical user interface*
- *CLI- command line interface*
- *Batch*

Skeleton of system:



Evolution of OS:

- no OS
- Uni programming (punch cards)
- batch processing
- multiprogramming environment with one CPU (all the programs are readily available in the memory and executed one by one)
- multi processing environment with more than one CPU (multiple programs are executed and one time where these multiple process will

be readily available in the memory and each process will be executed by separate CPU)

- time sharing (each process allocated sometime for the CPU execution it will follow like Round Robin approach)
- real time operating system (the schedule policy for the process based on the deadline i.e. when we need to complete the process)
- distributed OS (here the layers are distributed among multiple systems)
- mobile OS (example : Android)
- cloud OS (only the bootable file is available in the local system all the other files are held in the cloud)

components or modules of OS:

- process management
- memory management
- File system
- IO management
- Networking
- Security

Process management:

CC f1.c - o F1

After executing this particular command there will be a output file ./f1 available in the memory but the program file for this output file will be available only in secondary storage not the complete code file will be available in the secondary storage.

When a program gets address space it is called the process.

When a program file is loaded into the memory it is called the process.

A program is a passive entity and a process is an active entity.

The program when executed is called the process.

that are two types of process, namely :

- Background process $\Rightarrow$ no user interaction
- foreground process $\Rightarrow$ requires user interaction

We have some components; one such main component is called the PID.

The PID is known as the product id which is the unique ID it is generated by the OS itself.

PID 1 is known as init process or also known as root process.

PID 0 is known as the lazy process.

The reason to have multiple children for a process is to achieve parallelism.
or fun process

Orphan process:

The process with no parent , that will be adopted by init and then it will be killed by the init process.

Zombie process:

- The process that neither lives nor dies.
- The parent is alive and the child has completed the process but the parent is busy doing other tasks. So, the child process is called a zombie process.

PCB-Process Control Block:

- Each process will have a separate PCB.
- PCB will have all the details about the process it belongs to.
- When process is terminated or killed, the PCB is cleared.

Process table:

- User level
- Kernel level

The process table has all the processes as its entries.

The contents of PCB:

- 1) Pid
- 2) Registers
- 3) Parents id
- 4) Memory management related informations
- 5) Process state (imp)
- 6) Owner/user of the process
- 7) Time and date - when a process is created

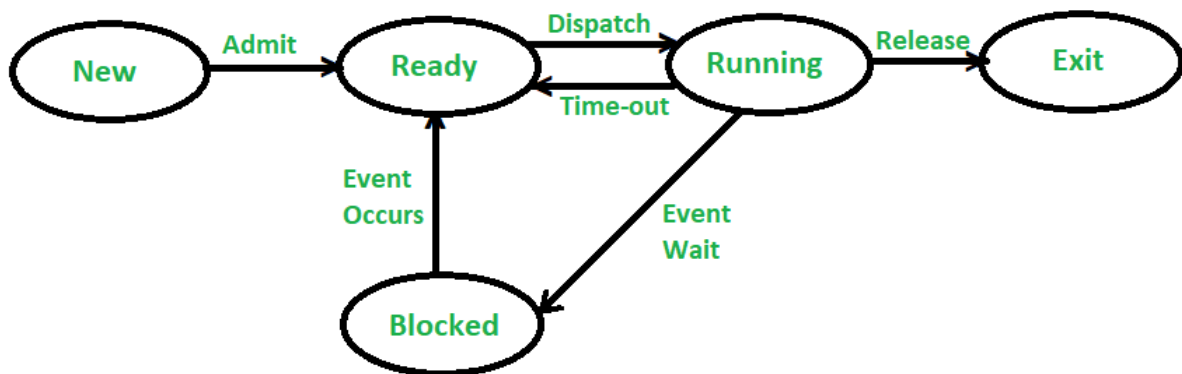
8) Scheduling related information

9) Priority of the process

Process state model:

- 5 state model (common)
- 7 state model

5 state model:



- All the new processes will be in the first state. (0-n process)
- A process is admitted when it clears all constraints (acquired all the resource required)
- Once the process came into the ready state (0-n process in the ready queue) i.e. The process is ready and waiting for the CPU.
- Dispatched or scheduled either 0 or 1 process will be in the running process as we are dealing with a single processor environment.

- While in the running state the process may be preempted and may go back to the ready state.
- If the process is completed then the process will be terminated or exited by releasing all the resources.
- But if it waits for an event to be triggered and completed such as IO process then it will be going into blocked state which has a dedicated queue called blocked or device queue. (O-n process)
- Note: all the queues will be of the same size.

Reason for process creation:

- Program execution
- Created by user (creation of child)
- interrupt

Reason for process killing:

- Program completes without an error.
- Killed by parent
- Killed by some other process or program.

Scheduling (move from one queue to other):

- Long-term
- Mid-term (no criteria to be followed by this)
- Short-term

New to ready: low (long term)

Ready to running: high (short term)

Blocked to ready: medium (mid term)

Therefore the frequency of process movement is high in the ready to running hence we need a scheduling. Hence if we optimize this by some means we will increase the cpu utilization and achieve maximum performance.

Criteria need to be followed by long-term schedule:

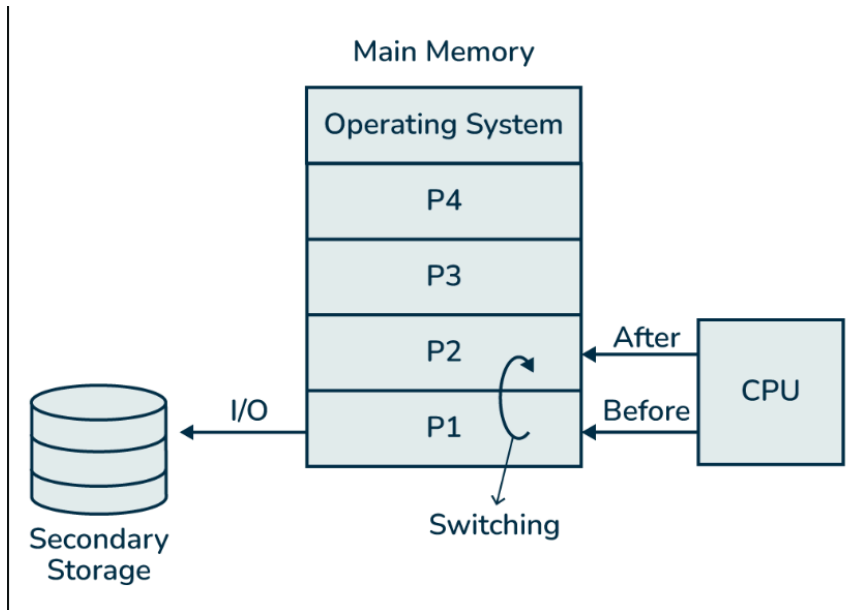
- Proper mixing of cpu bound and IO bound. Note: bound is the time taken or allocated to either CPU or IO

note : CPU bound: The process with more of the CPU process.

IO bound: The process with more of the IO process.

- The degree of multiprogramming needs to be maximized. But to maximize the degree of multiprogramming we need to follow a proper mixing of CPU bound & IO bound.

Note: degree of multiprogramming is no. of programming that can be kept readily available in the memory to be executed.



Criteria to be followed by short term scheduling:

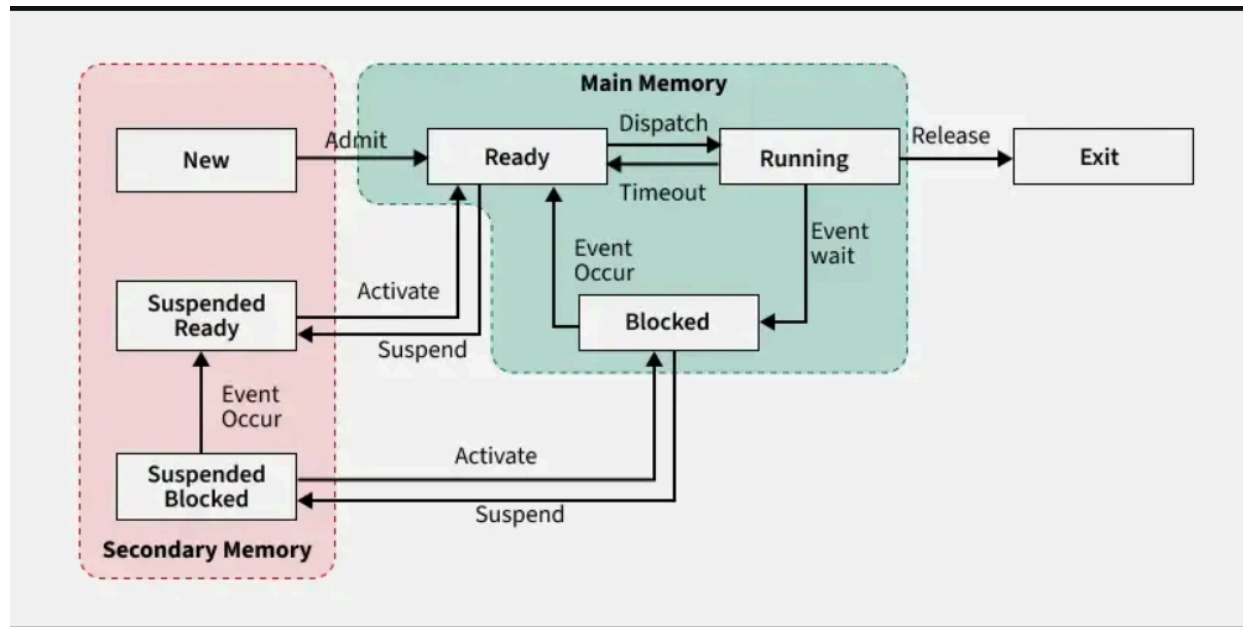
- *Maximize the cpu utilization.*
- *Minimize the waiting time.*
- *Minimize the turn around time. turn around time = waiting time + execution time*
- *Minimize the response time (response time: time taken to get the first response from the process to the user)*
- *Maximize the throughput.*

Note:

Whenever there is the secondary storage involved it is a costlier job as well as swapping. (swapping: Swapping in an operating system is a memory management technique that temporarily moves inactive or waiting processes

from physical RAM to secondary storage (disk) to free up space for other active processes)

7 state model:



The two suspend stages involve moving the process out of the main memory and placing it in secondary memory.

CPU scheduling Algorithm:

- Preemptive algorithms
- Non-preemptive algorithms

Context switching:

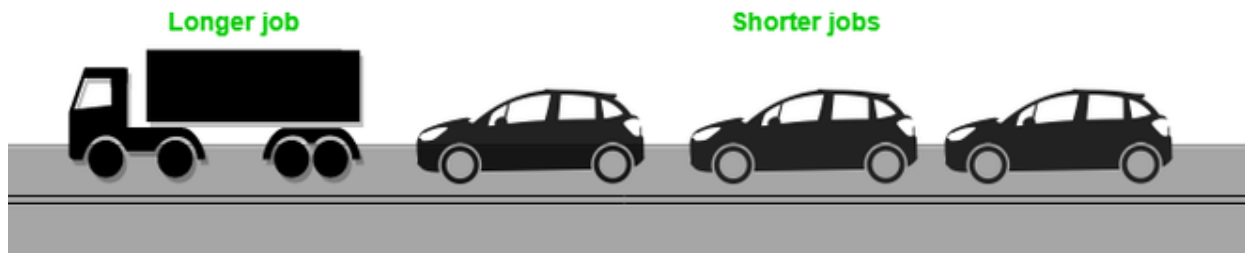
Moving one process to another process from the processor i.e. swapping the process that is being executed by the processor to another process.

Mode switch time < context switch time

Context switch time = mode switch time + other

Convoy effect:

- Whenever there is a smaller time process behind a large process which increases the waiting time.
- This effect is called the convoy effect.



Throughput:

No of process completed at a given time.

Gantt's diagram:

Generalized Activity Normalization Time Table (GANTT) chart is a type of chart in which a series of horizontal lines are present that show the amount of work done or production completed in a given period of time in relation to the amount planned for those projects. It is a horizontal bar chart developed by Henry L. Gantt (American engineer and social scientist) in 1917 as a production control tool. It is simply used for graphical representation of schedule that helps to plan in an efficient way, coordinate, and track some particular tasks in a project.

.....

Please refer to the notes, don't skip.

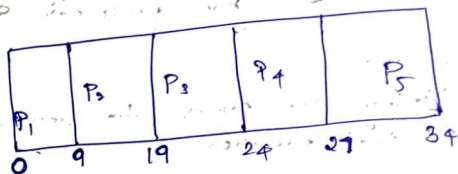
1) FIFO (First in First out)
 ↳ the process that comes first
 that process will be served
 non-preemptive first

Eg:

Process	Arrival time	Execution time
P ₁	0	9
P ₂	0	10
P ₃	0	5
P ₄	0	3
P ₅	0	7

Cpu bound
 CS=0

Gantt chart



Process | Waiting time

P ₁	0
P ₂	9
P ₃	19
P ₄	24
P ₅	27

$$\frac{(9+19+24+27)}{4} = 15.84$$
 Avg

$$P_1 = 0 + 9 = 09$$

$$P_2 = 9 + 10 = 19$$

$$P_3 = 19 + 5 = 24$$

$$P_4 = 24 + 3 = 27$$

$$P_5 = 27 + 7 = 34$$

$$113/5 = 22.6$$

at time 20 P_1, P_2 are completed
and P_3 has started hence
 P_1, P_2, P_3 has a response time but
no response from P_4, P_5

It is An unbiased algorithm.

Eg:

Process	A.T	E.T
P_1	0	9 (3+1+2+1+2)
P_2	2	10 (4+2+4)
P_3	4	5 (2+1+2)
P_4	6	3 (3)
P_5	8	7 (1+1+1+1+1)

W.T

$$P_1 = 0$$

$$P_2 = 7$$

$$P_3 = 15$$

$$P_4 = 18$$

$$P_5 = 19$$

E.T

$$P_1 = 0 + 9 = 9$$

$$P_2 = 7 + 10 = 17$$

$$P_3 = 15 + 5 = 20$$

$$P_4 = 18 + 7 = 25$$

$$P_5 = 19 + 7 = 26$$

$$\frac{9+17+20+25+26}{5} = 19$$

Convoy effect

* whenever there is a smaller time process behind large process will increase the waiting time.

A this ~~process~~ effect is called Convoy Effect.

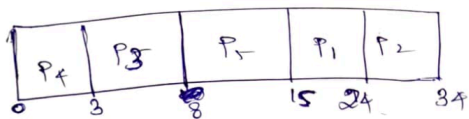
2) SJF (Shortest Job First Algorithm)

→ preemptive - ~~SPTN~~ SPTN

→ non-preemptive - SJF

Process	BT	AT
P ₁	9	0
P ₂	10	0
P ₃	5	0
P ₄	3	0
P ₅	7	0

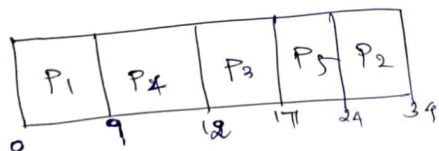
non-preemptive



W.T
 $P_1 = 15$
 $P_2 = 24$
 $P_3 = 3$
 $P_4 = 0$
 $P_5 = 8$
 $\frac{0}{5} = 0.1$

Preemptive (SRTF)

Process	Est	A-E
P ₁	9	0
P ₂	10	2
P ₃	5	5
P ₄	3	7
P ₅	7	8



P_1 97 0

P_2 10 0

P_3 54 2

P_4 3 8

P_5 7 4

P_1	P_3	P_4	P_3	P_1	P_5	P_2
0	2	3	6	10	17	24
						34

N.t

$$P_1 = 0 + 8 = 08$$

$$P_2 = 24 = 24$$

$$P_3 = 2 + 3 = 05$$

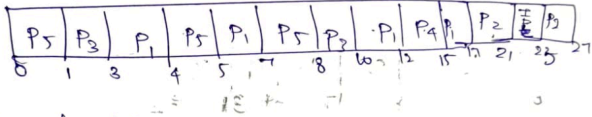
$$P_4 = 3 = 03$$

$$P_5 = 17 = 17$$

$$\frac{57}{5} = 11.4$$

- ① SRN - Shortest Remaining time next
- P_1 9 (3 + 1 + 2 + 1 + 2)
- P_2 10 (4 + 2 + 4)
- P_3 5 (2 + 1 + 2)
- P_4 3 (3)
- P_5 7 (1 + 3 + 1 + 1 + 1)

Preemptive:-



through put: no. of process completed at a given time.

Priority based Scheduling Algorithm

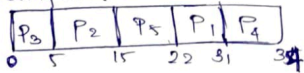
Preemptive

Non-preemptive

	AT	ET	Priority
P_1	0	9	3
P_2	0	10	1
P_3	0	5	0
P_4	0	3	4
P_5	0	7	2

(Lower value of the priority has higher priority)

non-preemptive:



Preemptive:

	AT	ET	Priority	
P ₁	0	9 → 7	3	X
P ₂	2	10 → 9	1	X
P ₃	3	5	0	X
P ₄	4	3	4	
P ₅	5	7	2	X

P ₁	P ₂	P ₃	P ₄	P ₅	P ₁	P ₄
0	2	3	8	17	24	31
						34

Disadvantages:
 * Starvation (Solved by Aging technique)

Round Robin algorithm:

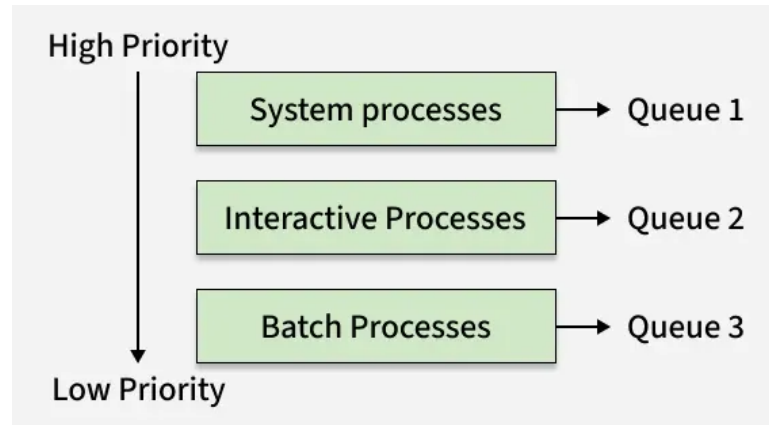
- For the time shared environment, the RR algorithm is best suited.
- Pure preemption
- Each process is allocated a particular time period for the CPU.
- The time period is called a time slice / time quanta.
- Try to minimize the response time.

If $t_q \rightarrow 0$: each process assumes that each process will have its own processor with low capacity.

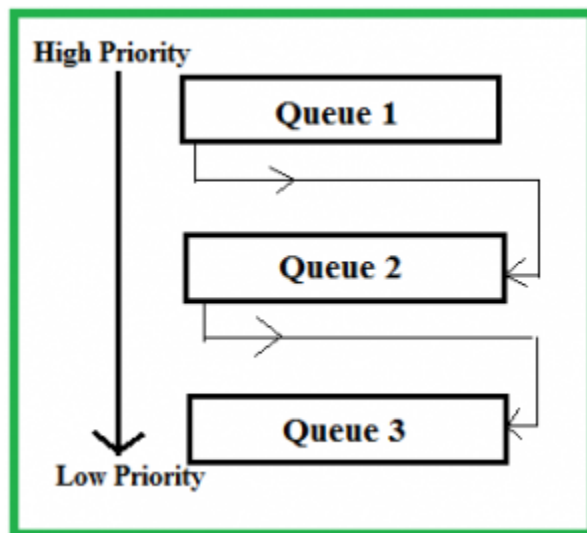
If $t_q \rightarrow \infty$: the strategy will act as a FIFO algorithm.

Multilevel queue scheduling algorithm:

- We can have multiple ready queue.

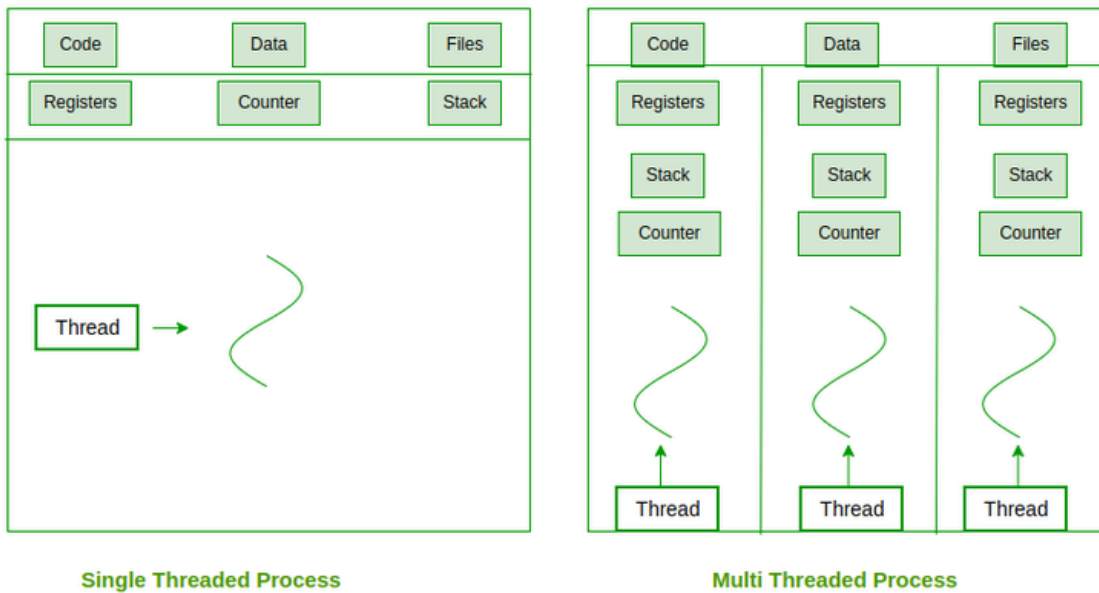


Multilevel feedback queue scheduling algorithm:



Practically we can't achieve 100% utilization of the CPU because of the context switch time.

Threads:



Inter process communication (IPC):

- *Communication between the two process without any common thing.*

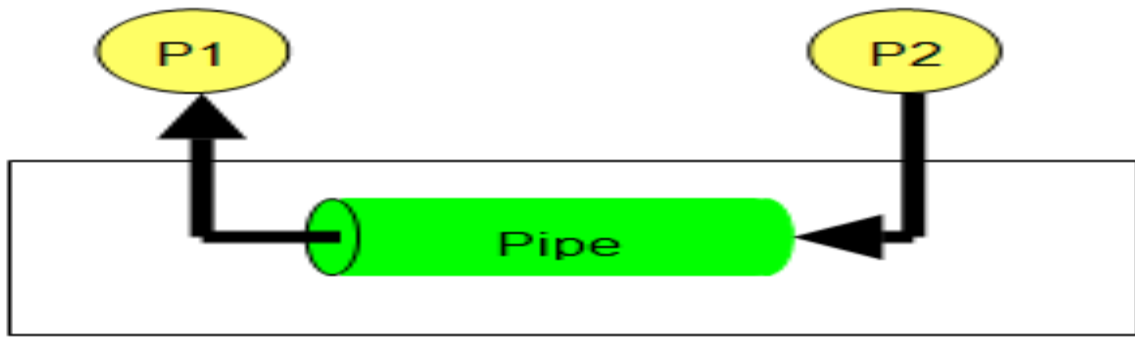
IPC-mechanisms:

- *Pipe (sender and receiver runs in same machine)*
- *Shared memory (sender and receiver runs in same machine)*
- *Message passing (sender and receiver runs in different machines)*

Pipe:

One-way pipe:

- *The processes using the one-way pipe are always synchronised.(i.e. when the sender is ready to send the data then the receiver should be ready to receive the data at the same time)*

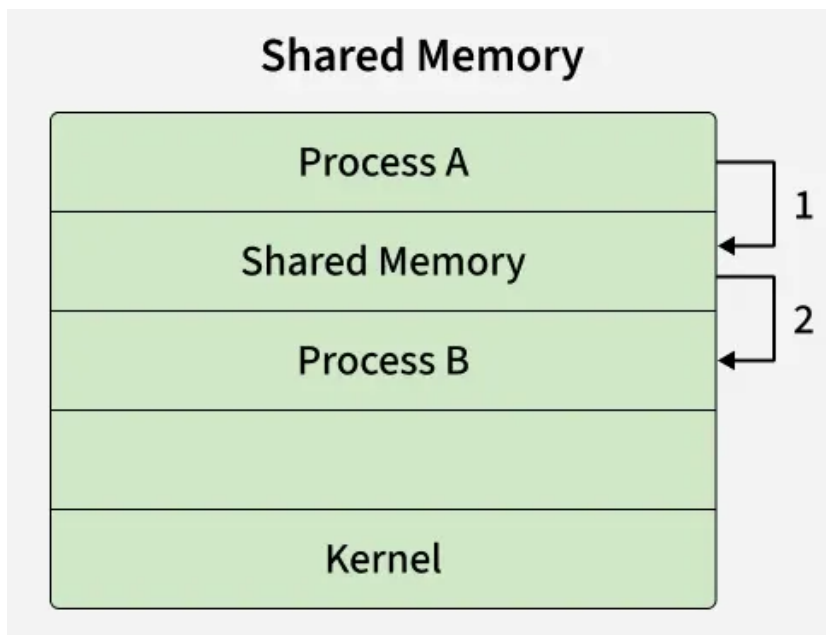


Two-way pipe:

- *Sending and receiving of the data will be in the same clock cycle.*
- *Writing and reading data into pipes involves kernel mode.*
- *Creating pipes is also kernel mode.*

Shared memory:

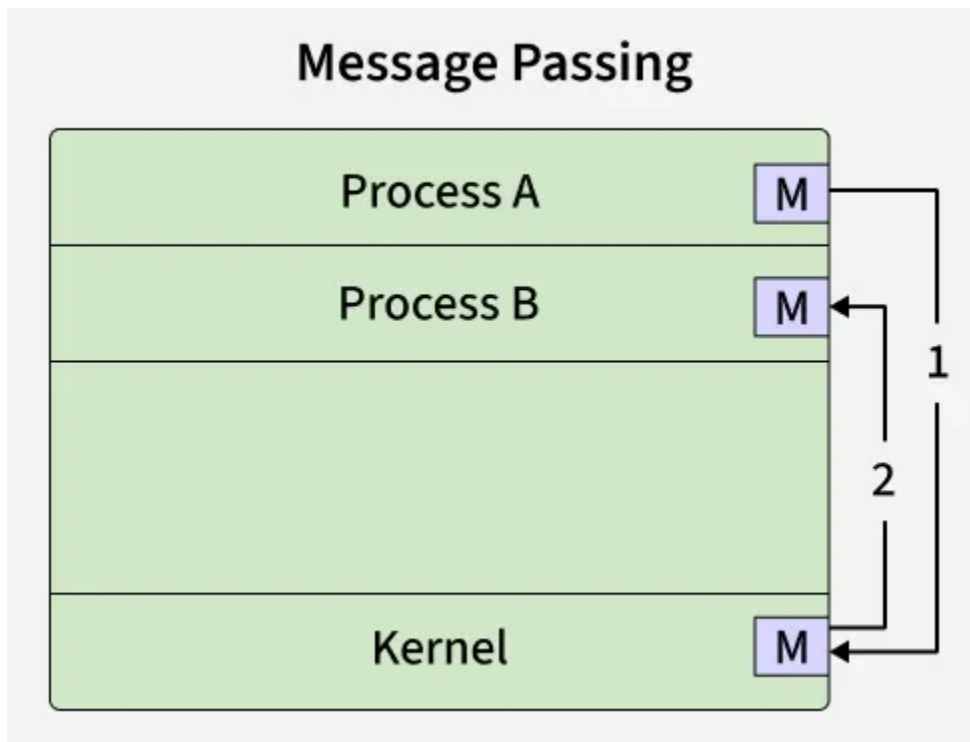
- *Asynchronous method.*



- The shared memory is the memory allocated to have the communication between process A and process B.
- The shared memory is allocated and mapped by the kernel.

Message passing:

- It is also an asynchronous method.

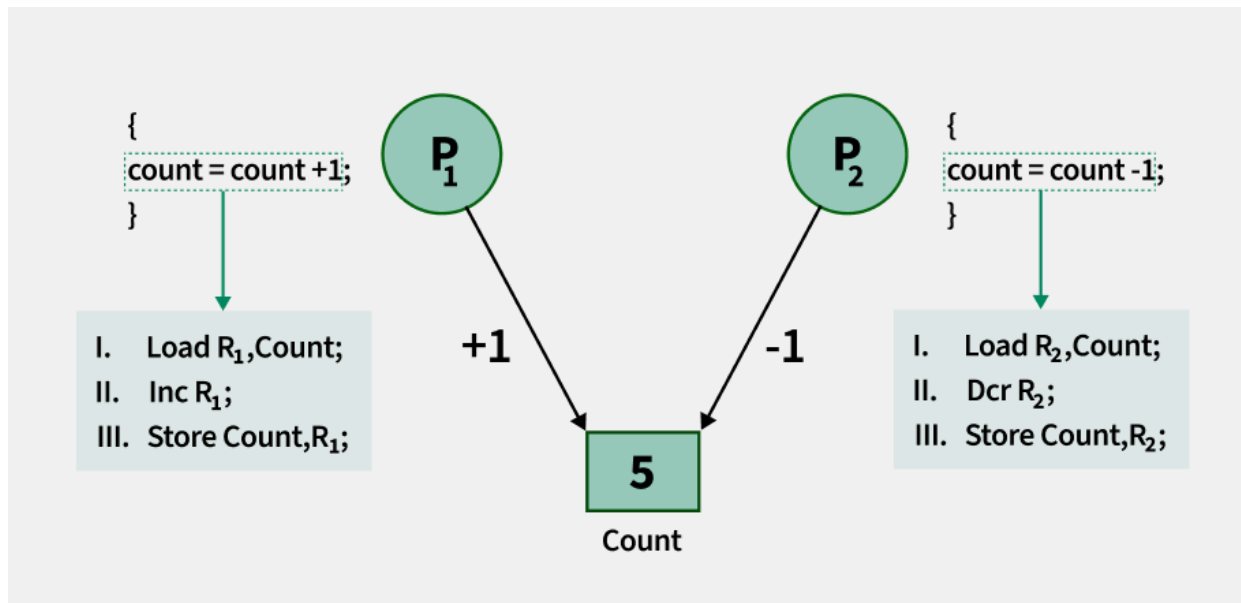


- Send and receive system calls are used here. They take place in kernel mode.

Different message:

- It is a typical client-server architecture.

Synchronization:



From the above image we can say that there will be some inconsistency due to the shared variable $count$ by both the processes.

Hence that resource (memory, variable, file...) is called a critical section or critical region.

The above image 🙌 denotes a condition called race condition. This is a condition that arises when two or more processes try to modify the same data at the same time.

Process synchronisation:

The synchronisation problem is also called the critical section problem.

Conditions to be followed by the solution to solve the process

synchronisation:

- No 2 processes can be there in the critical section at any time. (mutual exclusion).
- No assumptions about no CPUs and speed of CPU.
- No process executing a non critical region prevents another from entering into the critical region.
- No process should wait forever to enter into a critical region (no starvation).

Solutions:

- Hardware solution (processor level):
 - Disabling and enabling the interrupt
 - TSL (test and set lock commands)
- OS solution (kernel mode):
 - Semaphore
 - Monitor
- Software solutions (user mode i.e non-privileged instructions):
 - Using lock variable
 - Strict alternate
 - Peterson's rule

Using lock variable:

- Lock is common between the processes.

If $lock == 0 \Rightarrow$ open

If $lock == 1 \Rightarrow$ close

lock=0;

P1:

while($lock == 1$);

while($lock == 0$);

lock=1;

//critical section
section

lock=0;

P2:

lock=1;

//critical

lock=0;

Strict alternate:

turn=1;

P1:

while($turn == 2$);

while($turn == 1$);

//critical region

turn=2;

P2:

//critical region

turn=1;

Peterson's rule:

$turn=0;$
 $interest[]=\{F,F\};$

$P_0:$

$interest[0]=T;$

$turn=1;$

$while(turn==1 \ \&\& \ interest[1]==T);$

//critical section

$interest[0]=F;$

$P_1:$

$interest[1]=T;$

$turn=0;$

$while(turn==0 \ \&\& \ interest[0]==T);$

//critical section

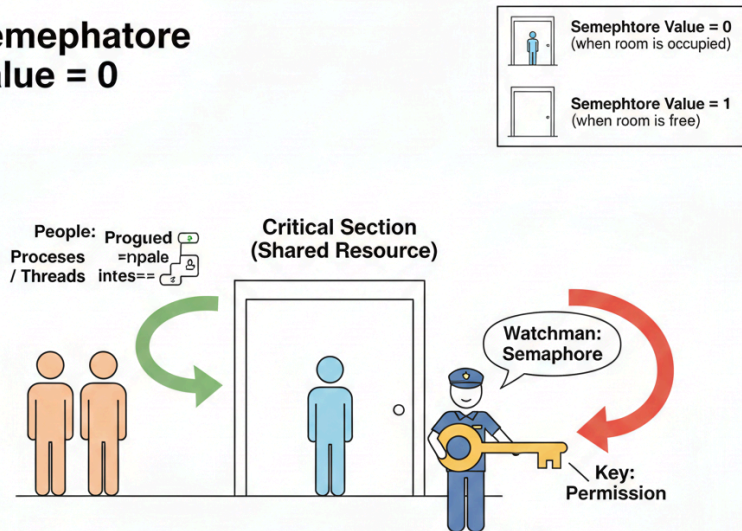
$interest[1]=F;$

Operating system solution:

- Semaphore

Semaphore is a tool provided by the OS itself.

Semaphore Value = 0



- Here comes the twist: semaphore is not a fancy name but a simple ordinary integer variable.
- But the catch is it can perform only 2 operations like wait and signal.
- Both the wait and signal are atomic operations.
 - When one process is performing any one of wait or signal other process can perform neither wait nor signal.

```
Semaphore s=3;//no of processes need to access the critical region
wait(s) // → asking for the key
//critical section
```

signal(s) // → returning the key

Void wait(semaphore s)

{

s--;

if(s < 0)

// block the process and place it in the semaphore queue

}

Void signal(semaphore s)

{

s++;

if(s <= 0)

// place it in ready queue and remove one process from the

queue

and allow the process to critical region

}

- *Wait and signal can be performed in any order*
- *We can perform wait and signal operation on two different semaphores.*
- *Eg:*

Semaphore s1=1;

Semaphore $s_2=0$;

P_1

`while(1){`

`wait(s1);`

`wait(s2);`

`printf("0");`

`printf("1");`

`signal(s2);`

`signal(s1);`

`}`

P_2

`while(1){`

`}`

We achieve a strict alternative by using semaphore variables.

Semaphor

|

Counting semaphore <-----|-----> binary

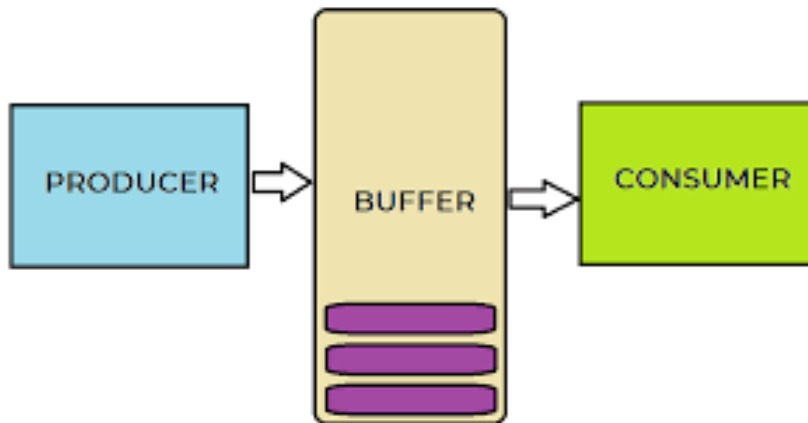
semaphore

- Counting semaphore takes value from $-\infty$ to $+\infty$
- Binary semaphore takes value either 0 or 1
 - They are also called as mutex semaphore
 - Used this to implement mutual exclusion

Classical synchronization problems:

- Producer - consumer problem
- Reader - writer problem
- Dining philosopher problem

1) Producer - consumer problem:



- The producer process will produce resources to the consumer process. Once the consumer process consumes the process the resource is time out
- If the buffer is empty, the consumer must be stopped.
- If the buffer is full, the producer must be stopped.
- Here the buffer is common between two processes hence it is the critical region.

Solution:

Semaphore mutex=1;

Semaphore empty=n; //size of the buffer

Semaphore filled=0;

Producer

wait(empty);

wait(filled);

wait(mutex);

wait(mutex);

insert_item(buffer);

remove(buffer);

Signal(mutex);

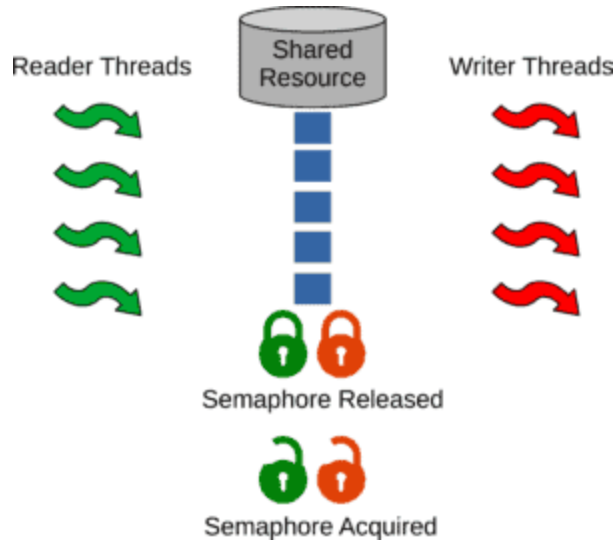
signal(mutex);

signal(filled);

signal(empty);

Consumer

2) Reader-writer problem:



Reader: who reads the data

Writer: who write the data

- *When one writer is in a critical section then at that time another writer or reader is not allowed.*
- *When one reader is in critical section then only readers are allowed but no writers are allowed.*

Solution:

Mutex m1=1,m2=1;

Int count=0; // ⇒ this is the critical section

Writer:

wait(m1);

//writing

signal(m1);

if(count==1)

Reader:

wait(m2);

count++;

wait(m1);

signal(m2);

//reading

wait(m2);

Count --;

if(count==0)

signal(m1);

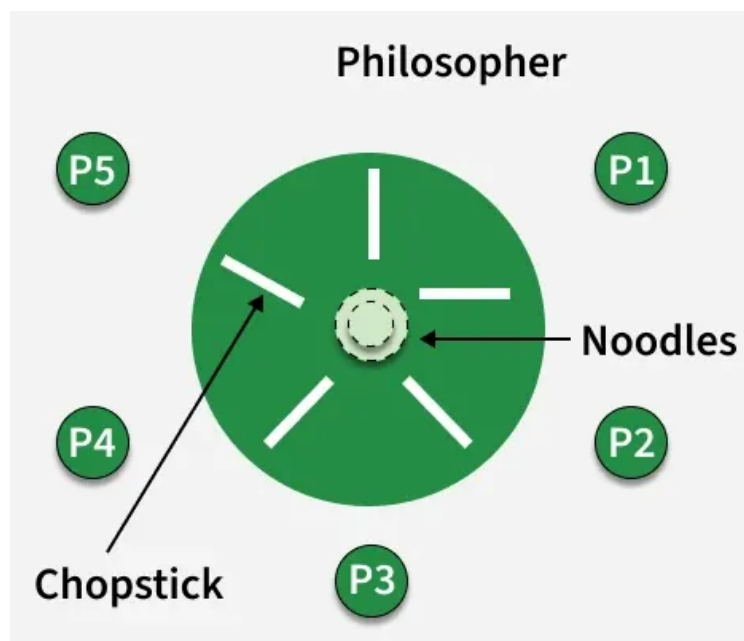
signal(m2);

Difference between reader-writer and producer-consumer process:

Reader-Writer vs Producer-Consumer Problem

Purpose	Reader-Writer	Producer-Consumer
Access control	Access control	Data flow
Shared Resource	Same data	Buffer
Concurrency	Multi readers	Buffer-based
Main Issue	Read-write conflict	Full / Empty
Example	Database	Print queue
Synchorization	RW lock	Empty / Full

3) Dining philosopher problem:



There are just 3 states:

- Thinking
- Hungry
- Eating

Solution:

semaphore chopstick[5] = {1,1,1,1,1};

Philosopher(i):

while(true) {

 think();

 wait(chopstick[i]); // pick left chopstick

 wait(chopstick[(i+1)%5]); // pick right chopstick

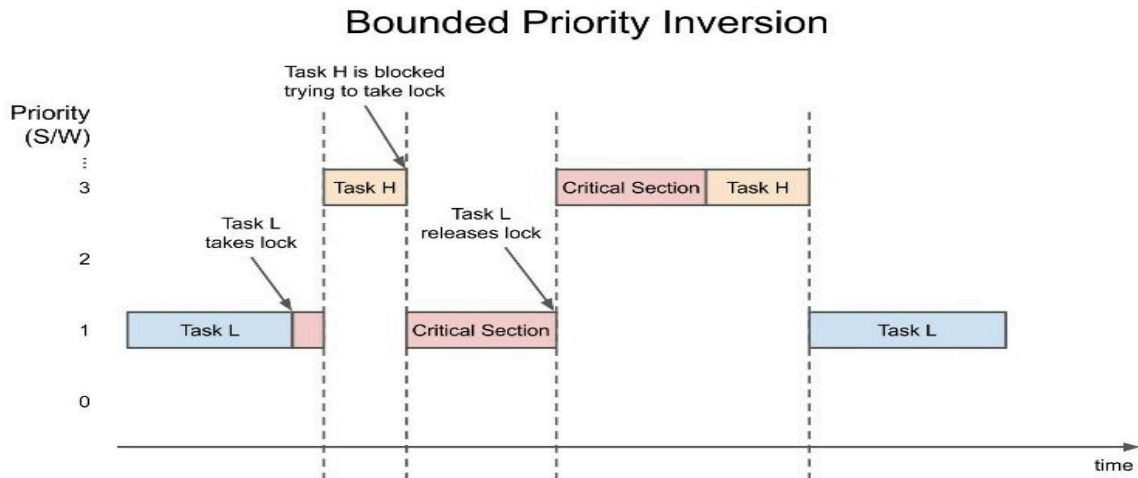
 eat();

 signal(chopstick[i]); // put left chopstick

 signal(chopstick[(i+1)%5]); // put right chopstick

}

4) Priority Inversion:



We have 3 processes with 3 different priorities namely:

- L (low)
- M (mid)
- H (high)

The order of arrival is L, M, H.

The sequence of events:

1. L acquires the mutex and enters its critical section.
2. H arrives and attempts to acquire the mutex but is blocked, waiting for L to release it.
3. M becomes ready to run and preempts L.

4. M executes until it completes its task.

5. L resumes, finishes its critical section, and releases the mutex.

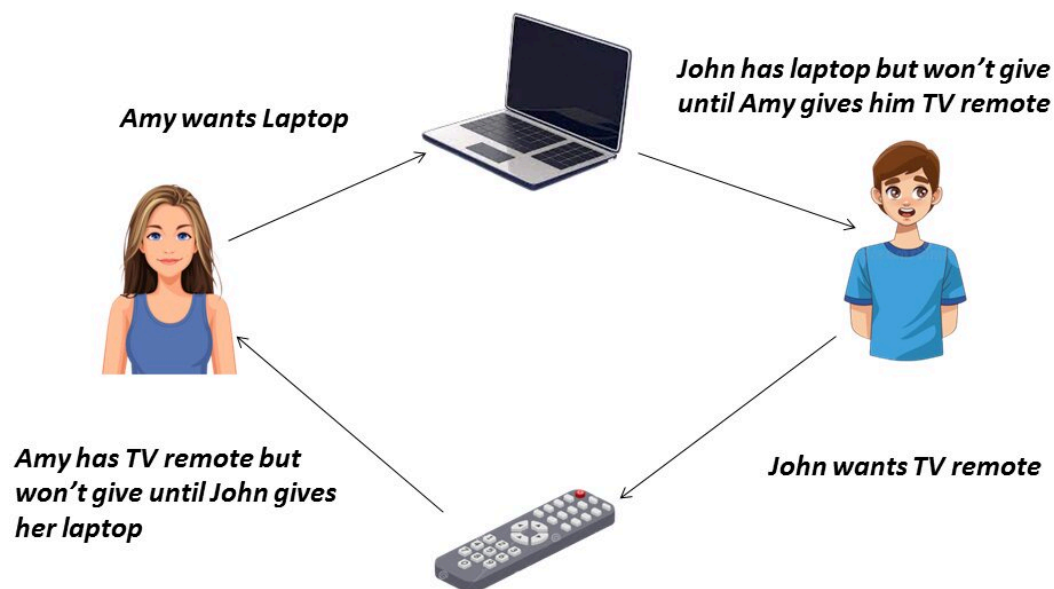
6. H acquires the mutex and enters its critical section.

The order of execution is M, H, L .

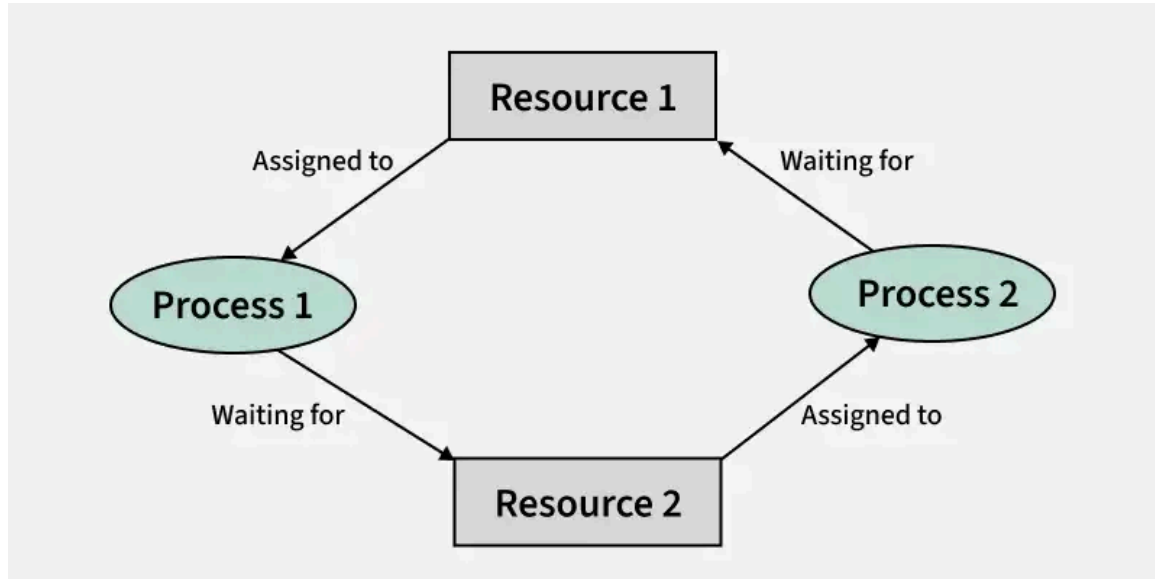
But we need H, M, L so we go for priority inversion protocol.

- Priority is temporarily inverted when a lower-priority process (p_1) is in the critical section and a higher-priority process (p_2) is blocked by it. In this scenario, the priorities of p_1 and p_2 are effectively swapped.

Dead lock:



Here Amy and John are two processes and the laptop and remote are the resources.



Legend:

- If the direction of the arrow is from the resource and towards the process then that means the resource is held by the process.
- If the direction of the arrow is from the process towards the resource then it means the process is requesting for the resource.

Safe sequence:

If the processes can be executed in a particular order without any problem then there is no dead-lock.

Dead lock definition:

- A set of processes is dead locked if each process in the set is waiting for an event that only another process in the set can cause.

Note:

- There is no efficient solution for dead locking.
- Most OS don't deal with dead-lock.
- Because, the loss due to dead lock $\ll$ the loss due to deadlock recovery

Resources

- Preemptable resource
- Non - preemptable resource

Methods to handle deadlock:

- Deadlock prevention
- Deadlock avoidance
- Deadlock recovery

Dead lock prevention:

- The designer of the OS designs the OS in a way that the deadlock never happens.

Dead lock avoidance:

- By carefully allocating resources like whenever allocating resources we should ensure that the dead-lock does not occur.

Dead lock recovery:

- The dead lock occurred but we will recover from that dead lock.

Conditions for dead lock:

- Mutual exclusion
- Hold and wait
- No preemption
- Process irreversible

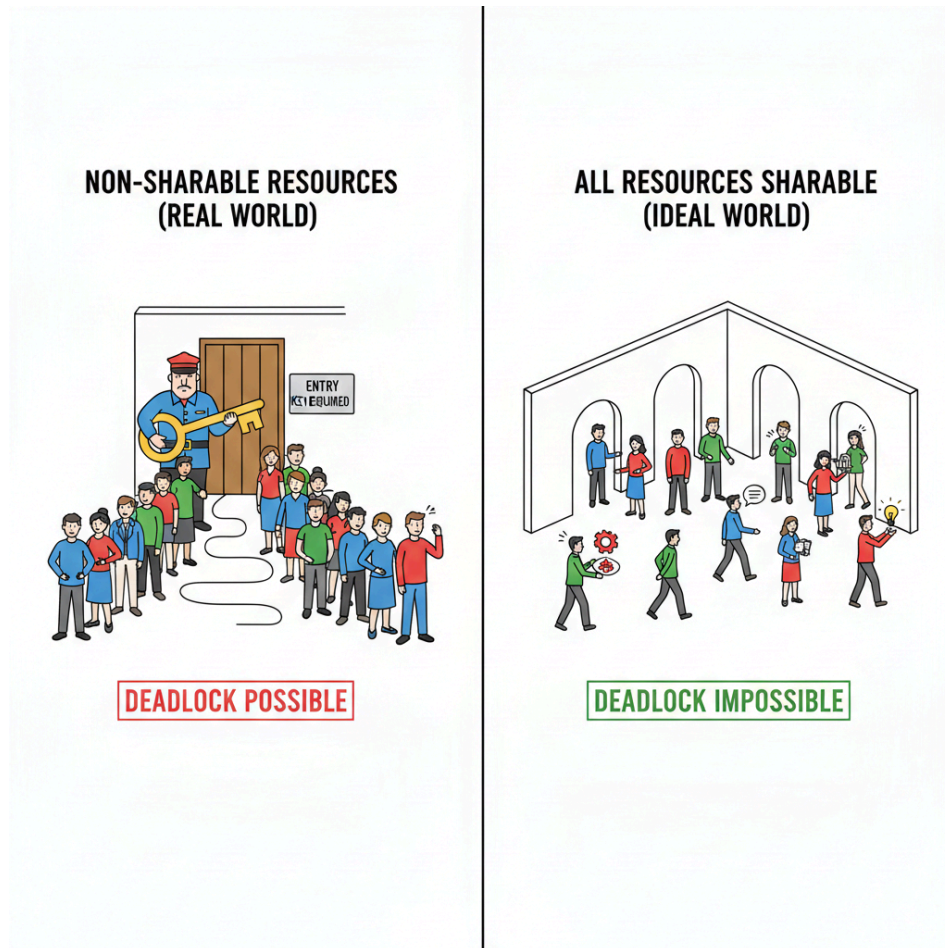
All the above conditions are called necessary conditions.

The sufficient condition is whenever there is a circular wait there is a dead lock, provided that there are only one instance of resources.

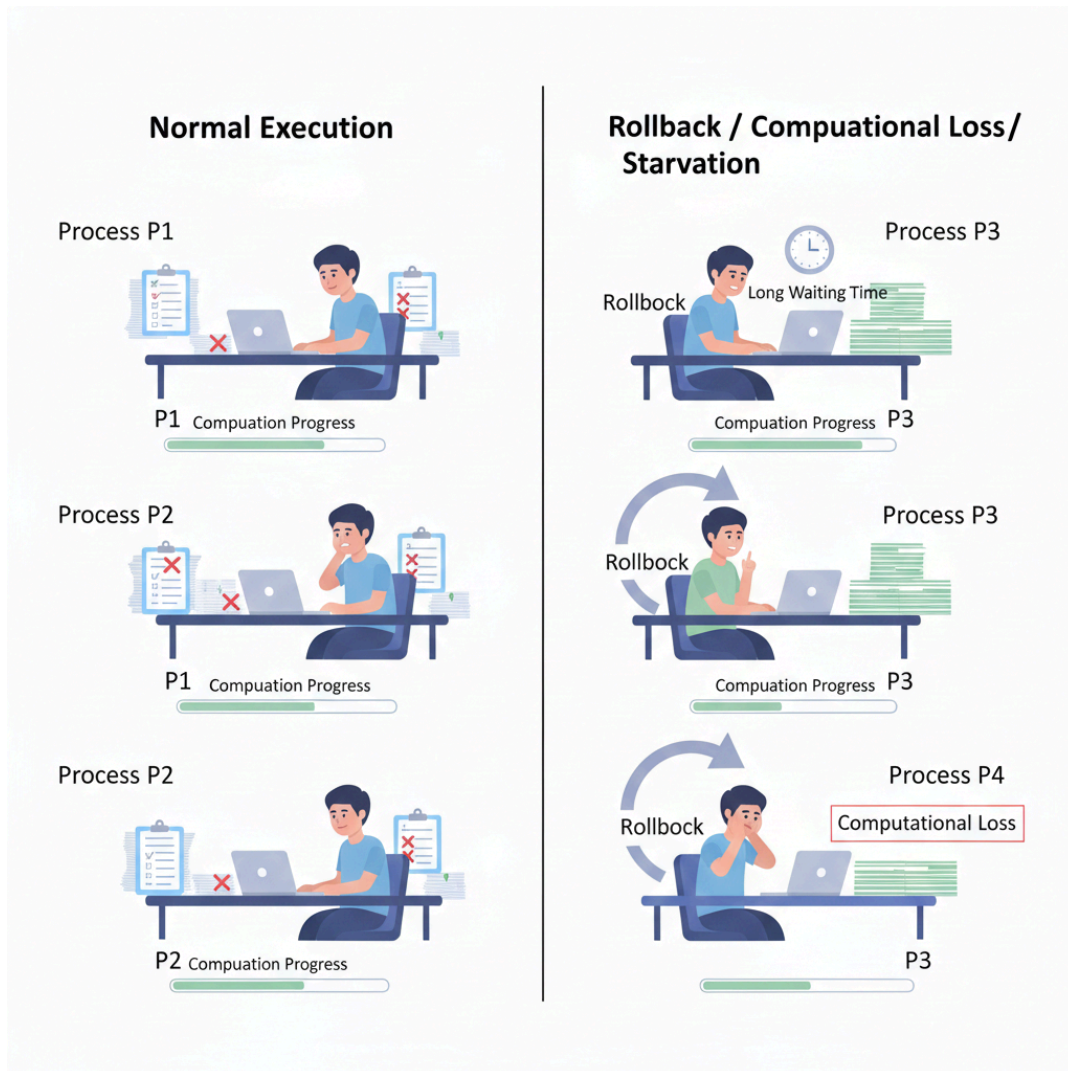
Note:

To prevent dead lock, we need to disallow any of the above conditions.

- Mutual exclusion can be disallowed by making all the resources sharable but practically we can't make it like that.

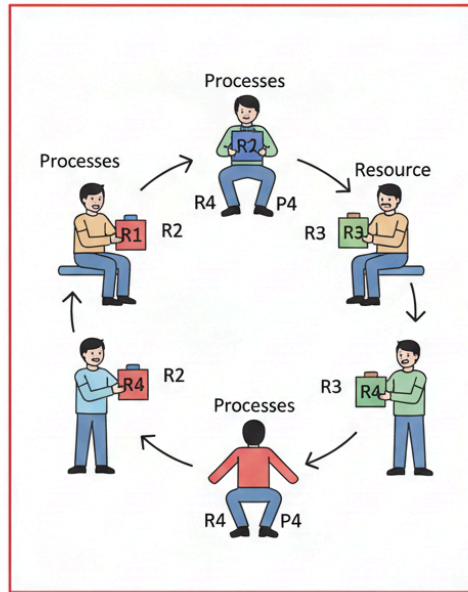


- Hold and wait can be disallowed by making the process to release all the resources before requesting for new resources but the consequence would be starvation and long waiting time.
- We can't make all the resources as preemptable.
- Computational loss and starvation due to roll back of the same processes when we try to make all processes reversible.



- *To disallow circular wait we arrange the resource in an order with respect to the resource id and all the process will request in the same order. But the waiting time is more and resource utilization is poor.*

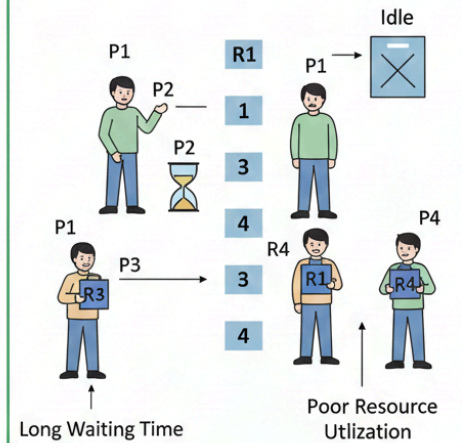
Circular Wait (Deadlock Possible)



DEADLOCK POSSIBLE

Circular Wait Prevented

Resources ordered by ID. Processes must request in increasing order.



DEADLOCK IMPOSSIBLE

Dead lock avoidance:

- Banker's algorithm:

Banker's Algorithm in C

Safe Resource Allocation



In Operating System



Avoids Deadlocks

Example



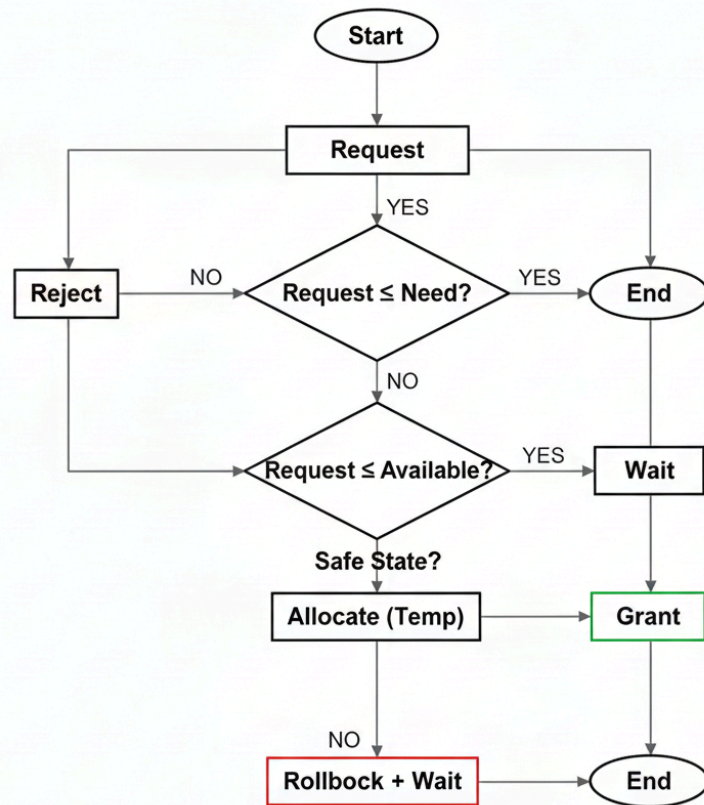
Requests Loan from Bank
EDUCBA



Banker's Algorithm Checks



Outcome
If Safe State: Loan Approved
If Unsafe State: Loan Denied



- Here the request is like asking loan to the bank
- Need is like the required money the cashier needs to give to other customers

Pseudo code:

If (Request $\leq$ Need && Request $\leq$ Available){

Allocate temporarily;

If Safe $\rightarrow$ Grant;

Else $\rightarrow$ Rollback;

}

Else

Wait;
